

PostgreSQL Production Guide for Senior Backend Interviews

1) Postgres Internals (Interview-Critical)

- Process-per-connection architecture.
- Shared buffers + WAL + background writer/checkpointer.
- MVCC for concurrent reads/writes.

Why MVCC matters:

- Readers do not block writers in many cases.
- Dead tuples accumulate and require VACUUM.

2) Transactions and Isolation

Isolation levels:

- Read Committed (default).
- Repeatable Read.
- Serializable.

Interview points:

- Anomalies prevented at each level.
- Serializable can abort transactions; app must retry.

Example retry pattern:

```
for attempt := 1; attempt <= 3; attempt++ {  
    err := runSerializableTx(ctx, db)  
    if err == nil {  
        break  
    }  
    if !isSerializationError(err) {  
        return err  
    }  
    time.Sleep(backoff(attempt))  
}
```

3) Indexing Deep Dive

Types:

- B-tree: default, equality/range.
- Hash: specialized equality.
- GIN: arrays/jsonb/full-text.
- GiST: geometric/range types.
- BRIN: huge append-heavy tables.

Best practices:

- Index for actual query patterns.

- Watch write overhead from over-indexing.
- Prefer composite indexes aligned with where/order usage.

4) Query Planning and Tuning

Tools:

- EXPLAIN (ANALYZE, BUFFERS).
- pg_stat_statements.
- auto_explain.

Interpretation basics:

- Seq Scan may be fine on small tables.
- Nested loop vs hash join vs merge join trade-offs.
- row estimate mismatch often indicates stale statistics.

High-value tuning actions:

- right indexes.
- query rewrite.
- updated statistics via ANALYZE.
- partitioning for very large tables.

5) Schema Design and Data Modeling

- Normalize for consistency, denormalize selectively for performance.
- Use proper constraints: PK, FK, unique, check.
- Use partial indexes for sparse predicates.

JSONB guidance:

- Good for flexible attributes.
- Still index keys used in filters.
- Avoid turning relational schema into opaque blob store.

6) Locking and Concurrency

Lock types:

- Row-level locks via FOR UPDATE.
- Table-level locks for DDL and some operations.

Deadlock handling:

- Postgres detects deadlocks and aborts one tx.
- Keep lock order consistent across code paths.

7) Replication and High Availability

- Streaming replication with primary + standbys.
- Sync vs async replication trade-offs.
- Failover orchestration (managed/unmanaged).

Interview expectation:

- discuss replication lag impact on read-after-write consistency.

8) Partitioning Strategy

- Range/list/hash partitioning.
- Good for time-series and large event tables.
- Enables partition pruning and maintenance efficiency.

Operational notes:

- partition management automation.
- index and constraint strategy across partitions.

9) Maintenance and Operations

9.1 VACUUM and Autovacuum

- Autovacuum prevents bloat and transaction ID wraparound.
- Poor autovacuum tuning can degrade performance.

9.2 Backup and Restore

- Base backup + WAL archiving for PITR.
- Regular restore drills are mandatory.

9.3 Connection Management

- Limit app connection count.
- Use connection pooling (pgBouncer/RDS Proxy).

10) Postgres in Go Services

Go patterns:

- context-aware queries.
- retries for transient/serialization failures.
- bounded transaction scope.
- explicit timeout per query class.

Example:

```
ctx, cancel := context.WithTimeout(ctx, 500*time.Millisecond)
defer cancel()
row := db.QueryRowContext(ctx, `select id, status from orders where id=$1`, id)
```

11) Performance Checklist

- slow query log/pg_stat_statements review.
- index hit ratio and cache health.
- lock wait monitoring.
- replication lag alarms.
- autovacuum and bloat tracking.

12) Postgres Interview Questions

1. Why can an index hurt performance?

- Extra write/update overhead and larger memory footprint.
2. How do you diagnose a slow query?
- EXPLAIN ANALYZE BUFFERS + stats + index and cardinality check.
3. What causes table bloat?
- MVCC dead tuples + delayed/insufficient vacuum.
4. How do you prevent connection storms?
- pool limits, queueing, and backpressure at app layer.
5. How do you plan safe schema migrations?
- backward-compatible changes, phased rollout, lock-impact aware DDL.

13) Migration and Rollout Best Practices

- Expand and contract migration pattern.
- Dual-write only when necessary and validated.
- Backfill in chunks with checkpointing.
- Canary rollout and rollback script ready.

14) Story Prompts

- Resolved major slow-query incident with indexing/query rewrite.
- Improved reliability through pooling and timeout strategy.
- Led migration with zero downtime and rollback safety.
- Reduced bloat and stabilized performance with autovacuum tuning.

15) SQL Snippets and Interview Explanations

15.1 EXPLAIN ANALYZE for Slow Query Investigation

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, created_at
FROM orders
WHERE customer_id = 42
ORDER BY created_at DESC
LIMIT 50;
```

Interview explanation:

- Compare estimated rows vs actual rows.
- If mismatch is large, stats may be stale or predicate selectivity misunderstood.

15.2 Composite Index Aligned with Query Pattern

```
CREATE INDEX CONCURRENTLY idx_orders_customer_created_desc
ON orders (customer_id, created_at DESC);
```

Interview explanation:

- Matches filter + sort path.
- CONCURRENTLY avoids long write lock, useful in production.

15.3 Worker Queue with SKIP LOCKED

```
BEGIN;

WITH next_job AS (
  SELECT id
  FROM jobs
  WHERE status = 'pending'
  ORDER BY created_at
  FOR UPDATE SKIP LOCKED
  LIMIT 1
)
UPDATE jobs
SET status = 'processing', picked_at = now()
WHERE id IN (SELECT id FROM next_job)
RETURNING id;

COMMIT;
```

Interview explanation:

- Multiple workers can safely pull without double-processing.

15.4 Retry Pattern for Serialization Failures

```
func runWithSerializableRetry(ctx context.Context, fn func(tx *sql.Tx) error) error {
    for attempt := 1; attempt <= 3; attempt++ {
        tx, err := db.BeginTx(ctx, &sql.TxOptions{Isolation: sql.LevelSerializable})
        if err != nil {
            return err
        }

        err = fn(tx)
        if err == nil {
            if cErr := tx.Commit(); cErr == nil {
                return nil
            }
            err = tx.Rollback()
            _ = err
        } else {
            _ = tx.Rollback()
        }

        if !isSerializationFailure(err) {
            return err
        }
        time.Sleep(time.Duration(attempt) * 50 * time.Millisecond)
    }
    return fmt.Errorf("too many retries")
}
```

Interview explanation:

- Serializable isolation is safest but requires application retry strategy.

15) L4 PostgreSQL Deep Drill

15.1 MVCC and Bloat

- Readers do not block writers due to MVCC snapshots.
- Dead tuples accumulate and require vacuum.
- Long-running transactions delay cleanup and increase bloat.

Interview signal:

- Mention autovacuum tuning, not only index creation.

15.2 Index Strategy Beyond Basics

- B-Tree for equality/range.
- GIN for JSONB and full text.
- BRIN for very large append-only time series.
- Partial indexes for skewed predicates.

Example:

```
CREATE INDEX CONCURRENTLY idx_orders_open
ON orders (created_at)
WHERE status = 'OPEN';
```

15.3 Locking and Contention Diagnosis

```
SELECT pid, username, state, wait_event_type, wait_event, query
FROM pg_stat_activity
WHERE state <> 'idle';
```

Interview explanation:

- At L4, you explain blocking chains, not only "database is slow".

15.4 Replication and Failover

- Physical replication for HA/read scaling.
- Logical replication for selective data movement and migrations.
- Define failover policy: automatic vs operator-driven with data loss trade-off.

15.5 Partitioning for Large Tables

- Time/range partitioning for retention and maintenance.
- Drop old partitions instead of deleting billions of rows.
- Ensure partition pruning is effective in query plans.

15.6 Senior Grilling Questions

1. Why did p99 write latency jump during autovacuum?
2. How do you migrate hot table schema with minimal lock risk?
3. What consistency risk appears after async failover?

4. Why can "more indexes" hurt write-heavy workloads?

16) L5 PostgreSQL Internals and Capacity Strategy

16.1 Capacity Planning Language for Interviews

Model with:

- write amplification (indexes + WAL),
- autovacuum throughput vs dead tuple creation,
- checkpoint cadence and IO pressure,
- replication lag bounds under peak writes.

16.2 Query Plan Regression Playbook

- Capture baseline plans for critical queries.
- Compare row-estimation error, join order drift, and index choice.
- Validate with `EXPLAIN (ANALYZE, BUFFERS)` before and after stats/index changes.

16.3 Schema Evolution at Scale

- Backward-compatible contract first.
- Expand -> backfill -> switch reads/writes -> contract.
- Long-running backfills throttled with observability and pause/resume controls.

17) PostgreSQL Incident Drills

17.1 Sudden Lock Contention

Triage:

- blocker PID and lock type,
- long transaction sources,
- DDL conflicts with OLTP paths,
- safe kill strategy and app retry behavior.

17.2 Replication Lag Crisis

Triage:

- write burst vs replica apply throughput,
- network/disk bottlenecks,
- query load on replica,
- failover safety thresholds.

17.3 Bloat and Vacuum Starvation

Triage:

- table/index bloat indicators,
- autovacuum settings mismatch,
- transaction age and wraparound risk,
- controlled maintenance windows.

18) L5 Grilling Questions (PostgreSQL)

1. How do you balance index coverage with write throughput?
2. Why does MVCC help reads but still create operational debt?
3. How do you prove partition strategy is helping, not hurting?
4. What is your rollback plan for a bad migration under live traffic?
5. How do you choose physical vs logical replication by use case?

19) 60-Minute PostgreSQL Revision Sprint

0-20 min:

- MVCC, locks, indexes, planner fundamentals.

20-40 min:

- replication/failover and partition strategy.

40-60 min:

- one real incident story + one migration story with measured outcomes.
-